

Implementacja i Optymalizacja Testu Pierwszości AKS w Języku C

Marcel Słotwiński

Projekt Zaliczeniowy

Idea i tło historyczne

- **Problem:** Czy $n \in PRIMES$?
- **Podział naiwny:** $O(\sqrt{n})$ – czas wykładniczy względem rozmiaru wejścia ($\log n$).
- **Przełom (2002):** Algorytm Agrawala, Kayala i Saxeny.
- **Status:** Pierwszy deterministyczny i bezwarunkowy test czasu wielomianowego.
- **Teoria:** Dowód, że $PRIMES \in P$.

Wielomianowy test Fermata

Tożsamość dla liczb pierwszych:

$$(X + a)^n \equiv X^n + a \pmod{n}$$

Redukcja stopnia wielomianu: Zastosowanie pierścienia ilorazowego

$\mathbb{Z}_n[X]/(X^r - 1)$:

$$(X + a)^n \equiv X^n + a \pmod{X^r - 1, n}$$

- Wielomianowy czas obliczeń dzięki binarnemu potęgowaniu.
- Dobór parametru r kluczowy dla złożoności.

Algorytm AKS krok po kroku

1. Sprawdź, czy n jest potęgą doskonałą ($n = a^b$).
2. Znajdź najmniejsze r takie, że $o_r(n) > \log^2 n$.
3. Jeśli $1 < \gcd(a, n) < n$ dla $a \leq r$, zwróć **ZŁOŻONA**.
4. Jeśli $n \leq r$, zwróć **PIERWSZA**.
5. Dla $a = 1$ do $\lfloor \sqrt{\varphi(r)} \log_2 n \rfloor$:
 - Test kongruencji $(X + a)^n \equiv X^n + a \pmod{X^r - 1, n}$.
6. Jeśli testy przejdą pomyślnie, zwróć **PIERWSZA**.

Wyzwania implementacyjne

- **Złożoność:** $O^{\sim}(\log^{\frac{15}{2}} n)$ – „wielomianowa”, ale z dużymi stałymi.
- **Problem $O(r^2)$:** „Szkolne” mnożenie wielomianów wąskim gardłem.
- **Zasoby:** Przy 32 bitach wielomian ma stopień ≈ 5000 .
- **Operacje:** Miliardy instrukcji GMP na każde potęgowanie.
- **Sprzęt:** R_{23} (74 bity) wymaga optymalizacji, by zakończyć test w czasie $< 1h$.

PREZENTACJA KODU

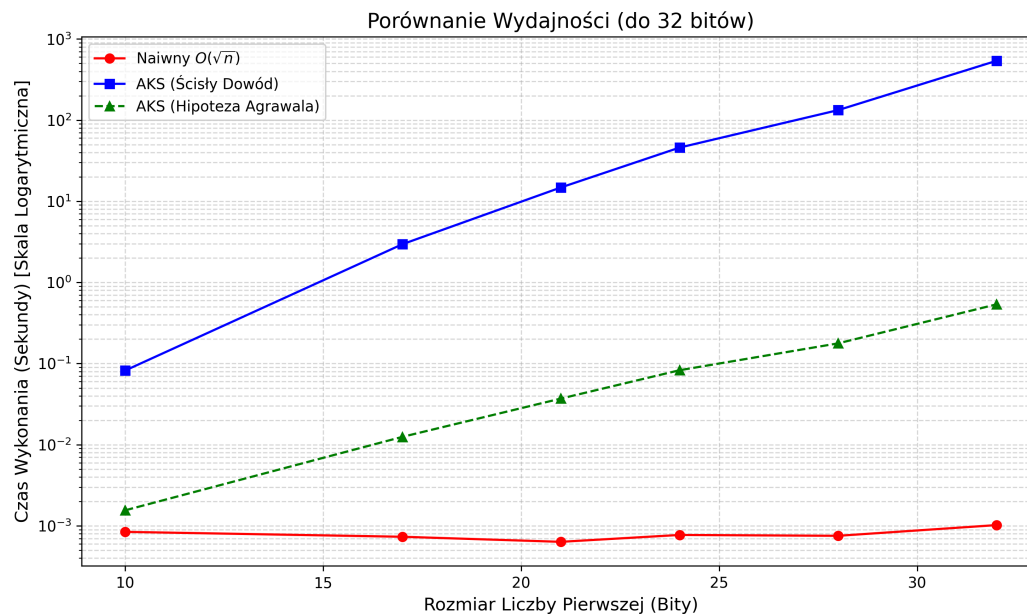
Struktury GMP, Leniwe Modulo, Metoda Babilońska

Kluczowe rozwiązania inżynierskie

- **Niejawne NWD:** Integracja sprawdzenia podzielności z pętlą szukania r .
- **Własności arytmetyki modularnej:** Akumulacja sum wielkich liczb (`mpz_addmul`) i redukcja na końcu.
- **Pierwiastek całkowitoliczbowy:** Czysto całkowitoliczbową metodą babilońska.
- **Unsigned int 128-bitowy:** Wykorzystanie `__uint128_t` do ochrony przed overflow.

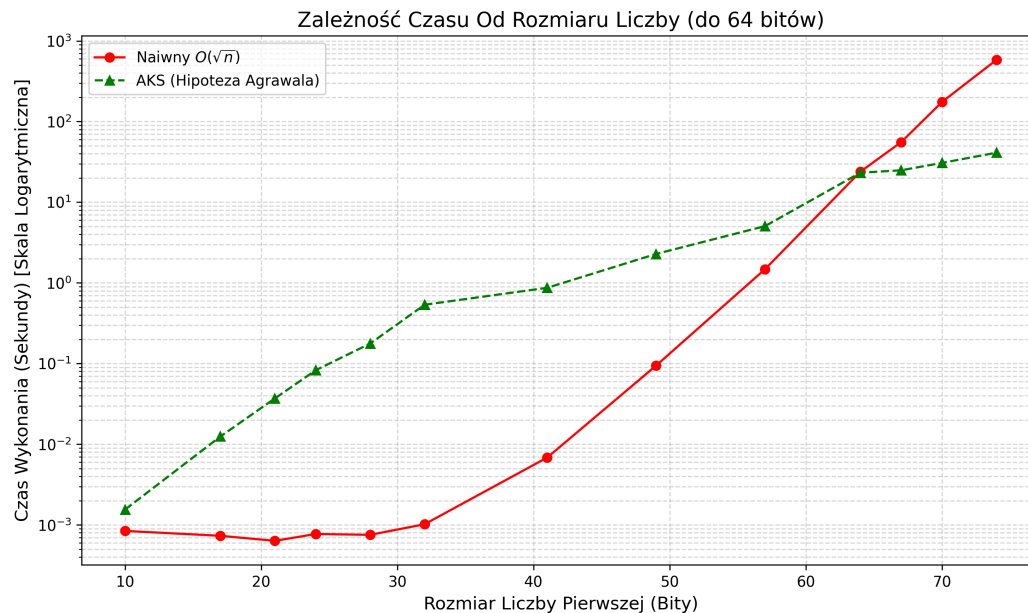
Wyniki pomiarów klasycznego AKS

- Drastyczny narzut pętli Kroku 5 (liczba świadków a).
- **Próg użyteczności:** Algorytm naiwny dominuje do ≈ 30 bitów.
- **Wynik krytyczny (32 bity):**
 - ok. 10 minut (AMD Ryzen 7 5800X).
- Skalowanie staje się niepraktyczne bez dodatkowych założeń.



Rysunek 1: AKS Strict vs Naive (log Y)

Przełom: Hipoteza Agrawala



$$(X - 1)^n \equiv X^n - 1 \pmod{X^r - 1, n}$$

- Redukcja limitu świadków do $a = 1$.
- **Crossover point:** 64 bity (≈ 23 sekundy dla obu metod).
- **Zwycięstwo:** Przy >64 bitach test naiwny kapituluje.

Rysunek 2: Zwycięstwo wielomianowości
($\log Y$)

Wnioski

1. **Teoria:** AKS udowadnia, że pierwszość jest problemem łatwym obliczeniowo.
2. **Praktyka:** Klasyczny AKS wymaga optymalizacji (FFT) do realnych zastosowań.
3. **Synergia:** Połączenie optymalizacji C (Lazy Modulo) i matematycznych (Agrawal) pozwala na testowanie liczb 128-bitowych na komputerze PC.
4. **Konkluzja:** Czas wielomianowy zawsze wygrywa z wykładniczym po przekroczeniu bariery $n \approx 2^{64}$.